

TECHNICAL TRAINING COURSE

Dispatching Requests to Methods

JAX-RS 2.x | Request Routing & Method Selection

Java 11 · Tomcat 9.0 · Jersey 2.x



Topics Covered: @ApplicationPath · @Path · HTTP Method Annotations
Sub-resource Locators · Annotation Inheritance & Overriding

Platform: Java 11 LTS | Tomcat 9.0 | JAX-RS 2.1

Namespace: javax.ws.rs.* (NOT jakarta.ws.rs.*)

Data format: XML — @Produces(APPLICATION_XML) / @Consumes(APPLICATION_XML)

Duration: 1 Day (8 hours)

Level: Intermediate — Java Developer

Table of Contents

Table of Contents	2
1. The ApplicationPath	4
1.1 Learning Objectives	4
1.2 The Three-Part URI	4
1.3 @ApplicationPath in Depth	4
1.4 Effect on Request Dispatching	5
1.5 Multiple Applications in One WAR	5
1.6 web.xml Alternative	6
1.7 Review Questions	6
2. The @Path Annotation	7
2.1 Learning Objectives	7
2.2 Class-Level and Method-Level @Path	7
2.3 URI Template Variables	7
2.4 Regex-Constrained Template Variables	8
2.5 The Request Matching Algorithm	8
2.6 @Path Design Best Practices	8
2.7 Review Questions	9
3. The HTTP Method Annotations	10
3.1 Learning Objectives	10
3.2 Standard HTTP Method Annotations	10
3.3 Selecting Methods by HTTP Verb	10
3.4 Content Negotiation with @Produces and @Consumes	11
3.5 Response Codes by Convention	11
3.6 Custom HTTP Method Annotations	12
3.7 Review Questions	12
4. Sub-resource Locators	13
4.1 Learning Objectives	13
4.2 Two Kinds of Sub-resource	13
4.3 Sub-resource Method vs Locator — Side by Side	13
4.4 The Sub-resource Class	14
4.5 CDI Injection in Sub-resource Locators	14
4.6 Locator Chains	15
4.7 When to Use a Locator vs a Method	15
4.8 Review Questions	15
5. Annotation Inheritance and Overriding	16
5.1 Learning Objectives	16
5.2 JAX-RS Annotation Inheritance Rules	16

5.3 Class-Level Inheritance — Interface.....	16
5.4 Method Annotation Override — All or Nothing.....	17
5.5 Class Hierarchy Annotation Inheritance	18
5.6 The Interface Pattern — Benefits	18
5.7 Common Pitfalls.....	19
5.8 Review Questions	19
Appendix — Dispatch Decision Flowchart	20

4. If `getClasses()/getSingletons()` return non-empty sets, uses those; otherwise scans the classpath for `@Path` and `@Provider` classes.

1.4 Effect on Request Dispatching

The application path is the first filter in the dispatch chain. An incoming request is routed to a JAX-RS Application only if its URI starts with the context root + application path.

Incoming URI	Context Root	@ApplicationPath	Match?
/myapp/api/products	/myapp	/api	Yes — dispatched to ShopApplication
/myapp/admin/products	/myapp	/api	No — no Application matches /admin
/myapp/api/	/myapp	/api	Yes — root of the application
/otherapp/api/products	/myapp	/api	No — wrong context root

1.5 Multiple Applications in One WAR

```
import javax.ws.rs.ApplicationPath;
import javax.ws.rs.core.Application;

@ApplicationPath("/api/v1")
public class ApiV1Application extends Application {
    @Override public java.util.Set<Class<?>> getClasses() {
        return java.util.Set.of(V1ProductResource.class,
            V1OrderResource.class);
    }
}

@ApplicationPath("/api/v2")
public class ApiV2Application extends Application {
    @Override public java.util.Set<Class<?>> getClasses() {
        return java.util.Set.of(V2ProductResource.class,
            V2OrderResource.class);
    }
}

@ApplicationPath("/internal")
public class InternalApplication extends Application {
    @Override public java.util.Set<Class<?>> getClasses() {
        return java.util.Set.of(HealthResource.class, MetricsResource.class);
    }
}
```

Rules for multiple applications

Each Application must have a UNIQUE `@ApplicationPath` value.
 Providers registered in one Application are NOT shared with another.
 Shared providers (e.g. a `CorsFilter` or `JaxbContextResolver`) must be registered individually in every Application that needs them.

Two Applications with the same path cause a deployment error.

1.6 web.xml Alternative

When `@ApplicationPath` is absent, or when you need Tomcat-specific configuration, register the Servlet explicitly in web.xml. Use the Servlet 4.0 namespace and the javax.ws.rs.Application init-param name:

```
<!-- web.xml - Servlet 4.0 namespace for Tomcat 9 -->
<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
    http://xmlns.jcp.org/xml/ns/javaee/web-app_4_0.xsd"
  version="4.0">
  <servlet>
    <servlet-name>JerseyServlet</servlet-name>
    <servlet-class>
      org.glassfish.jersey.servlet.ServletContainer
    </servlet-class>
    <init-param>
      <!-- javax.ws.rs.Application - NOT jakarta.ws.rs.Application -->
      <param-name>javax.ws.rs.Application</param-name>
      <param-value>com.example.api.ShopApplication</param-value>
    </init-param>
    <load-on-startup>1</load-on-startup>
  </servlet>
  <servlet-mapping>
    <servlet-name>JerseyServlet</servlet-name>
    <url-pattern>/api/*</url-pattern>
  </servlet-mapping>
</web-app>
```

Priority rule

If BOTH `@ApplicationPath` is present AND a web.xml Servlet mapping is configured, the web.xml mapping takes precedence for that Application.

1.7 Review Questions

1. A WAR is deployed at /shop. The Application has `@ApplicationPath("/api")`. A resource has `@Path("/orders")`. What is the complete URL to reach a `@GET` method on that resource?
2. What happens when two Application classes share the same `@ApplicationPath` value?
3. You remove `@ApplicationPath` from your Application class and add nothing else. What URL pattern will Jersey use, if any?

2. The @Path Annotation

2.1 Learning Objectives

- Apply @Path at class and method level to define resource URIs
- Use URI template variables, regex constraints, and multi-segment templates
- Understand how the runtime selects the best matching resource and method
- Design clean, RESTful URI hierarchies using @Path

2.2 Class-Level and Method-Level @Path

```
import javax.ws.rs.*;
import javax.ws.rs.core.*;

@Path("/orders")                // class-level path
public class OrderResource {
    @GET                        // full path: /orders
    @Produces(MediaType.APPLICATION_XML)
    public java.util.List<Order> list() { ... }

    @GET @Path("/{id}")        // full path: /orders/{id}
    @Produces(MediaType.APPLICATION_XML)
    public Order get(@PathParam("id") String id) { ... }

    @GET @Path("/{id}/status") // full path: /orders/{id}/status
    @Produces(MediaType.TEXT_PLAIN)
    public String getStatus(@PathParam("id") String id) { ... }

    @POST @Path("/{id}/lines") // full path: /orders/{id}/lines
    @Consumes(MediaType.APPLICATION_XML)
    @Produces(MediaType.APPLICATION_XML)
    public Response addLine(
        @PathParam("id") String id, OrderLine line) { ... }
}
```

2.3 URI Template Variables

```
// Simple variable - matches any non-slash sequence
@GET @Path("/{id}") @Produces(MediaType.APPLICATION_XML)
public Order get(@PathParam("id") String id) { ... }

// Multiple variables in a single path expression
@GET @Path("/{year}/{month}/{day}") @Produces(MediaType.APPLICATION_XML)
public java.util.List<Order> byDate(
    @PathParam("year") int year,
    @PathParam("month") int month,
    @PathParam("day") int day) { ... }

// Automatic type conversion - JAX-RS converts the String to int
@GET @Path("/{id}") @Produces(MediaType.APPLICATION_XML)
public Product get(@PathParam("id") int id) { ... }
// Type conversion failure -> 404 Not Found
// GET /products/abc - 'abc' cannot convert to int -> 404
```

2.4 Regex-Constrained Template Variables

```
// Only matches numeric IDs — GET /products/42
@GET @Path("{id: \\d+}") @Produces(MediaType.APPLICATION_XML)
public Product getById(@PathParam("id") int id) { ... }

// Matches SKU codes — GET /products/WIDGET-001
@GET @Path("{sku: [A-Z]+-\\d+}") @Produces(MediaType.APPLICATION_XML)
public Product getBySku(@PathParam("sku") String sku) { ... }

// Greedy match — captures slashes — GET /files/a/b/c.txt
@GET @Path("files/{path: .+}")
public Response getFile(@PathParam("path") String path) { ... }

// Literal suffix — GET /reports/2024-summary.pdf
@GET @Path("{name: [^/]+}\\..pdf")
public Response getPdf(@PathParam("name") String name) { ... }
```

2.5 The Request Matching Algorithm

When a request arrives, the runtime applies a deterministic algorithm to select the best-matching resource class and method. The algorithm has three phases:

4. Identify candidate root resource classes whose class-level `@Path` matches the request URI
5. Within matching classes, find method-level `@Path` templates that match the remaining URI
6. Among all matching methods, apply tiebreaker rules to select exactly one

Term	Definition
Literal characters	More literal characters in the template wins. <code>/orders/active</code> beats <code>/orders/{id}</code> .
Template variables	Fewer template variables wins. <code>/orders/{id}/status</code> beats <code>/orders/{a}/{b}</code> .
Regex constraints	Templates with regex win over plain templates. <code>/orders/{id:\\d+}</code> beats <code>/orders/{id}</code> .
Media type specificity	<code>@Produces("application/xml")</code> beats <code>@Produces("*/*")</code> for an XML Accept header.

Practical Rule — Literals Beat Variables

Always remember: a more literal (more specific) path wins over a template.
`/orders/pending` wins against `/orders/{id}` for the request `GET /orders/pending`
 This means you can have a 'special' path and a generic `{id}` path on the same resource.

2.6 @Path Design Best Practices

- Use nouns, not verbs: `/orders` not `/getOrders` or `/fetchOrder`
- Use plural nouns for collections: `/orders` not `/order`
- Hierarchy reflects containment: `/orders/{id}/lines` reflects that lines belong to an order
- Use hyphens for multi-word segments: `/line-items` not `/lineItems` or `/line_items`
- Keep paths lowercase: `/products/{id}` not `/Products/{Id}`

- Avoid deep nesting beyond 3 levels — it becomes hard to reason about

2.7 Review Questions

7. Given `@Path("/catalogue")` on the class and `@Path("{category}/{id}")` on a method, what is the full path template?
8. A request arrives for GET `/products/featured`. You have methods with `@Path("{id}")` and `@Path("featured")`. Which matches and why?
9. You need a method that matches `/reports/2024/Q1/summary.pdf` and extracts `'2024/Q1/summary'` as one string. Write the `@Path` annotation.
10. What HTTP status code does the runtime return when a `@PathParam` cannot be converted to the declared type?

3. The HTTP Method Annotations

3.1 Learning Objectives

- Map each JAX-RS HTTP method annotation to its HTTP semantics
- Understand idempotency and safety and why they matter for API design
- Use `@Produces` and `@Consumes` to participate in content negotiation
- Create custom HTTP method annotations using `@HttpMethod`
- Return appropriate HTTP status codes via the Response builder

3.2 Standard HTTP Method Annotations

Annotation	HTTP Method	Safe?	Idempotent?	Typical Use
<code>@GET</code>	GET	Yes	Yes	Retrieve a resource or collection
<code>@POST</code>	POST	No	No	Create a resource or trigger an action
<code>@PUT</code>	PUT	No	Yes	Full replacement of a resource
<code>@PATCH</code>	PATCH	No	No	Partial update of a resource
<code>@DELETE</code>	DELETE	No	Yes	Remove a resource
<code>@HEAD</code>	HEAD	Yes	Yes	Same as GET but no response body
<code>@OPTIONS</code>	OPTIONS	Yes	Yes	Describe allowed methods; CORS pre-flight

Term	Definition
Safe	A method is safe if it has no observable side effects on the server. Clients and caches may freely retry safe methods.
Idempotent	A method is idempotent if calling it N times has the same effect as calling it once. Crucial for retry logic after network failures.

3.3 Selecting Methods by HTTP Verb

A resource class can handle multiple verbs on the same path. All imports use `javax.ws.rs.*`:

```
import javax.ws.rs.*;
import javax.ws.rs.core.*;

@Path("/products/{id}")
public class ProductItemResource {

    @GET // GET /products/{id}
    @Produces(MediaType.APPLICATION_XML)
    public Product get(@PathParam("id") String id) { ... }

    @PUT // PUT /products/{id}
    @Consumes(MediaType.APPLICATION_XML)
```

```

    @Produces(MediaType.APPLICATION_XML)
    public Product update(
        @PathParam("id") String id,
        Product updated) { ... }

    @PATCH // PATCH /products/{id}
    @Consumes(MediaType.APPLICATION_XML)
    public Response patch(
        @PathParam("id") String id,
        ProductPatch patch) { ... } // @XmlRootElement bean

    @DELETE // DELETE /products/{id}
    public void delete(@PathParam("id") String id) { ... }

    @HEAD // HEAD /products/{id} - same as GET,
    no body
    @Produces(MediaType.APPLICATION_XML)
    public Response head(@PathParam("id") String id) {
        Product p = get(id);
        return Response.ok()
            .header("X-Resource-Version", p.getVersion())
            .build();
    }
}

```

3.4 Content Negotiation with @Produces and @Consumes

@Produces and @Consumes participate in method selection as tiebreakers when multiple methods match the same verb and path. For XML and plain-text negotiation:

```

import javax.ws.rs.*;
import javax.ws.rs.core.*;

@GET @Path("/{id}")
@Produces(MediaType.APPLICATION_XML)
public Product getXml(@PathParam("id") String id) { ... }

@GET @Path("/{id}")
@Produces(MediaType.TEXT_PLAIN)
public String getSummary(@PathParam("id") String id) { ... }

// Client:  Accept: application/xml → calls getXml()
// Client:  Accept: text/plain → calls getSummary()
// Client:  Accept: text/html → 406 Not Acceptable
// Client:  Accept: */* → calls getXml() (first @Produces
// listed)

// @Consumes refines selection on Content-Type of inbound request body.
// If no method consumes the incoming Content-Type → 415 Unsupported Media
// Type.

```

3.5 Response Codes by Convention

Method	Success Code	Notes
@GET (found)	200 OK	Body contains the resource representation

@GET (not found)	404 Not Found	Resource does not exist
@POST (create)	201 Created	Location header must point to new resource
@POST (action)	200 OK	Action processed; body contains result
@PUT / @PATCH	200 OK	Body contains updated representation
@DELETE	204 No Content	No body expected on successful delete
@HEAD	200 OK	Same headers as GET; no body
Any (bad input)	400 Bad Request	Malformed request or validation failure
Any (wrong type)	415 Unsupported Media Type	Content-Type not accepted
Any (no match)	406 Not Acceptable	No method produces accepted media type
Any (wrong verb)	405 Method Not Allowed	PATH exists but not for that verb

3.6 Custom HTTP Method Annotations

JAX-RS allows you to create custom HTTP method annotations. Import `@HttpMethod` from `javax.ws.rs` (NOT `jakarta.ws.rs`):

```
// Define the custom annotation
import javax.ws.rs.HttpMethod;
import java.lang.annotation.*;

@Target({ElementType.METHOD})
@Retention(RetentionPolicy.RUNTIME)
@HttpMethod("PURGE") // tells JAX-RS this is the HTTP PURGE method
public @interface PURGE {}

// Use it in a resource class
@Path("/cache/{key}")
public class CacheResource {
    @PURGE
    public Response purge(@PathParam("key") String key) {
        cacheService.invalidate(key);
        return Response.noContent().build();
    }
}
```

3.7 Review Questions

11. Explain the difference between safe and idempotent. Give one HTTP method that is idempotent but not safe.
12. A client sends `DELETE /products/P001` twice. The second call finds the product already deleted. Should the server return 404 or 204? Justify your answer using idempotency.
13. What status code does JAX-RS return when the request URI matches a resource path but the HTTP method does not match any annotated method?
14. Write a `@HEAD` method for a product resource that returns a `Content-Length` header equal to the XML size of the product without sending the body.

4. Sub-resource Locators

4.1 Learning Objectives

- Define a sub-resource locator and distinguish it from a sub-resource method
- Implement locators that delegate request handling to a separate class
- Handle the CDI injection limitation in locator-returned instances
- Design hierarchical URI structures using locator chains
- Understand when locators add value over simple sub-resource methods

4.2 Two Kinds of Sub-resource

Term	Definition
Sub-resource method	A method with <code>@Path</code> AND an HTTP verb annotation (<code>@GET</code> , <code>@POST</code> , etc.). Handles the request directly. Part of the root resource class.
Sub-resource locator	A method with <code>@Path</code> but NO HTTP verb annotation. Returns an object. The runtime introspects that object and dispatches the request to it.

The key distinction is that a locator does not handle the request itself — it produces a handler. The JAX-RS runtime then calls the returned object's annotated methods to complete dispatch.

4.3 Sub-resource Method vs Locator — Side by Side

```
import javax.ws.rs.*;
import javax.ws.rs.core.*;

// — Sub-resource METHOD —————
@Path("/orders")
public class OrderResource {
    // Has @GET — handles the request directly
    @GET @Path("/{id}/status")
    @Produces(MediaType.TEXT_PLAIN)
    public String getStatus(@PathParam("id") String id) {
        return orderService.findById(id).getStatus().name();
    }
}

// — Sub-resource LOCATOR —————
@Path("/stores")
public class StoreResource {
    @javax.inject.Inject StoreService storeService;
    // No @GET, @POST, etc. — returns a handler object
    @Path("/{storeId}/inventory")
    public InventoryResource locateInventory(
        @PathParam("storeId") String storeId) {
        Store store = storeService.find(storeId);
        if (store == null) throw new NotFoundException();
        return new InventoryResource(store);
    }
}
```

4.4 The Sub-resource Class

```
import javax.ws.rs.*;
import javax.ws.rs.core.*;

// InventoryResource — no @Path at class level
public class InventoryResource {
    private final Store store;
    public InventoryResource(Store store) { this.store = store; }

    // Handles: GET /stores/{storeId}/inventory
    @GET @Produces(MediaType.APPLICATION_XML)
    public java.util.List<InventoryItem> listAll() {
        return store.getInventory();
    }

    // Handles: GET /stores/{storeId}/inventory/{sku}
    @GET @Path("{sku}") @Produces(MediaType.APPLICATION_XML)
    public InventoryItem getBySku(@PathParam("sku") String sku) {
        return store.getInventory().stream()
            .filter(i -> sku.equals(i.getSku()))
            .findFirst()
            .orElseThrow(NotFoundException::new);
    }

    // Handles: PUT /stores/{storeId}/inventory/{sku}
    @PUT @Path("{sku}")
    @Consumes(MediaType.APPLICATION_XML)
    @Produces(MediaType.APPLICATION_XML)
    public InventoryItem update(
        @PathParam("sku") String sku,
        InventoryItem item) {
        return store.updateInventory(sku, item);
    }
}
```

4.5 CDI Injection in Sub-resource Locators

```
import javax.ws.rs.*;
import javax.ws.rs.container.ResourceContext;

@Path("{storeId}/inventory")
public class InventoryResource {
    @PathParam("storeId") String storeId,
    @Context ResourceContext rc) { // inject ResourceContext
        Store store = storeService.find(storeId);
        if (store == null) throw new NotFoundException();
        InventoryResource resource = new InventoryResource();
        rc.initResource(resource); // triggers CDI injection
        resource.setStore(store); // set state after injection
        return resource;
    }

    // InventoryResource can now receive CDI injection:
    public class InventoryResource {
        @javax.inject.Inject InventoryService inventoryService;
        private Store store;
        public void setStore(Store s) { this.store = s; }
        // ...
    }
}
```

4.6 Locator Chains

```
// Level 1: /api/tenants/{tid}
@Path("/tenants")
public class TenantResource {
    @Path("/{tid}")
    public TenantScopeResource scope(@PathParam("tid") String tid) {
        Tenant t = tenantService.find(tid);
        if (t==null) throw new NotFoundException();
        return new TenantScopeResource(t);
    }
}

// Level 2: /api/tenants/{tid}/projects/{pid}
public class TenantScopeResource {
    private final Tenant tenant;
    public TenantScopeResource(Tenant t){ this.tenant = t; }
    @Path("projects/{pid}")
    public ProjectResource project(@PathParam("pid") String pid) {
        return new ProjectResource(tenant, pid);
    }
}

// Level 3: /api/tenants/{tid}/projects/{pid}/tasks
public class ProjectResource {
    @GET @Path("tasks") @Produces(MediaType.APPLICATION_XML)
    public java.util.List<Task> listTasks() { ... }
}
```

4.7 When to Use a Locator vs a Method

Use a Sub-resource Method when...	Use a Sub-resource Locator when...
The logic is simple and belongs in the root class	The handler class is complex enough to deserve its own file
There are only 1–2 extra operations on the sub-path	Many operations need to be grouped under the sub-path
No CDI injection is needed in the sub-resource	The sub-resource needs its own injected services
The hierarchy is shallow (one level)	The hierarchy is deep (two or more levels)

4.8 Review Questions

- What is the defining characteristic that makes a method a sub-resource locator rather than a sub-resource method?
- Describe the CDI injection problem with locator-returned instances and explain how ResourceContext solves it.
- Design a URI hierarchy for a university system with /departments/{id}/courses/{id}/students. Identify which levels should use locators and which should use methods.

5. Annotation Inheritance and Overriding

5.1 Learning Objectives

- Understand the annotation inheritance rules defined in the JAX-RS specification
- Apply annotations through interface hierarchies and class hierarchies
- Understand why method-level annotations do NOT inherit and how to work around this
- Override inherited annotations selectively at the method level
- Design annotation-driven service interfaces for testability and substitutability

5.2 JAX-RS Annotation Inheritance Rules

Term	Definition
Class annotations	ARE inherited. A subclass that does not redeclare a class-level <code>@Path</code> inherits it from its superclass.
Interface annotations	ARE inherited. If a class implements an interface annotated with <code>@Path</code> , the class inherits that annotation.
Method annotations	Are NOT automatically inherited. If a subclass overrides a method, it must redeclare ALL JAX-RS annotations — none are inherited from the superclass or interface method.
Partial override	If a subclass method redeclares ANY JAX-RS annotation, ALL annotations from the parent method are ignored — partial override is not supported.

Critical Rule — All or Nothing on Methods

JAX-RS method annotation inheritance is all-or-nothing.

If you override a method in a subclass and add even ONE JAX-RS annotation, you must redeclare ALL JAX-RS annotations from the parent method.

There is no merging or partial inheritance of method-level annotations.

5.3 Class-Level Inheritance — Interface

```
import javax.ws.rs.*;
import javax.ws.rs.core.*;

// Service interface — declares the REST contract
@Path("/products")
@Produces(MediaType.APPLICATION_XML)
@Consumes(MediaType.APPLICATION_XML)
public interface ProductService {
    @GET
    java.util.List<Product> listAll();

    @GET @Path("/{id}")
    Product findById(@PathParam("id") String id);

    @POST
    Response create(Product product, @Context UriInfo uriInfo);
}
```

```

}

// Implementation – inherits @Path, @Produces, @Consumes from the interface
public class ProductResource implements ProductService {
    @javax.inject.Inject ProductRepository repo;

    // @GET is inherited from the interface method – DO NOT redeclare
    @Override
    public java.util.List<Product> listAll() {
        return repo.findAll();
    }

    // @GET @Path("{id}") inherited – DO NOT redeclare
    @Override
    public Product findById(String id) {
        return repo.findById(id);
    }

    @Override
    public Response create(Product product, UriInfo uriInfo) {
        Product saved = repo.save(product);
        return Response.created(
            uriInfo.getAbsolutePathBuilder().path(saved.getId()).build()
                .entity(saved).build());
    }
}

```

5.4 Method Annotation Override — All or Nothing

```

// Base resource class
@Path("/reports")
public class ReportResource {
    // Base method: GET /reports, returns XML
    @GET
    @Produces(MediaType.APPLICATION_XML)
    public java.util.List<Report> list() {
        return reportService.findAll();
    }
}

// Subclass – wants to also support CSV
@Path("/admin/reports") // override class-level @Path
public class AdminReportResource extends ReportResource {

    // CORRECT – redeclare ALL method annotations
    @Override
    @GET
    @Produces({MediaType.APPLICATION_XML, "text/csv"}) // add CSV
    public java.util.List<Report> list() {
        return super.list();
    }

    // WRONG – only redeclares @Produces; @GET is lost → method not visible
    // @Override
    // @Produces({MediaType.APPLICATION_XML, "text/csv"})
    // public List<Report> list() { ... } // ← NOT a JAX-RS method!
}

```

5.5 Class Hierarchy Annotation Inheritance

```
import javax.ws.rs.*;
import javax.ws.rs.core.*;
import javax.xml.bind.annotation.*;

// JAXB-annotated health status bean
@XmlRootElement(name = "health")
@XmlAccessorType(XmlAccessType.FIELD)
public static class HealthStatus {
    @XmlElement public String status = "UP";
    public HealthStatus() {}
}

// Base class with @Path and @Produces
@Path("/base")
@Produces(MediaType.APPLICATION_XML)
public abstract class BaseResource {
    @GET @Path("health")
    public HealthStatus health() {
        return new HealthStatus(); // →
    }
}

// Inherits @Path("/base") and @Produces(APPLICATION_XML)
public class ChildResource extends BaseResource {
    // /base/health is still accessible via inherited method
}

// Overrides @Path but keeps @Produces(APPLICATION_XML)
@Path("/v2/products")
public class ProductResourceV2 extends BaseResource {
    // /v2/products/health works; /base/health no longer served by this class
}
```

5.6 The Interface Pattern — Benefits

- Testability — unit tests can use mock implementations of the interface
- Substitutability — swap implementations without changing the contract
- Documentation — the interface serves as a clean, scannable API contract
- Proxy generation — tools like MicroProfile REST Client generate client proxies from annotated interfaces
- Multiple implementations — different classes can serve different Application registrations

```
// MicroProfile REST Client — generates a JAX-RS client from the same
// interface
@org.eclipse.microprofile.rest.client.annotation.RegisterRestClient(
    baseUrl = "http://product-service")
public interface ProductService {
    @GET @Path("/products/{id}")
    @Produces(MediaType.APPLICATION_XML)
    Product findById(@PathParam("id") String id);
}

// Use as a typed HTTP client
@Inject
@org.eclipse.microprofile.rest.client.inject.RestClient
```

```
ProductService productService;  
  
Product p = productService.findById("P001"); // makes an HTTP GET call
```

5.7 Common Pitfalls

Term	Definition
Forgetting to redeclare @GET	Subclass method loses its verb annotation → treated as a locator, not a method.
Expecting @Path to merge	Subclass @Path("/admin") does NOT append to parent @Path("/base") — it replaces it entirely.
Parameter annotations	@PathParam, @QueryParam on interface method parameters ARE inherited. Do not redeclare them in the implementing class.
@Provider inheritance	@Provider on a superclass is NOT inherited. Each provider class must have its own @Provider annotation.

5.8 Review Questions

18. A class implements an interface with @Path("/widgets") and @GET on the listAll() method. The implementing class has no JAX-RS annotations at all. What URI will a GET request need to match to invoke listAll()?
19. A subclass overrides a method from its parent that has @GET, @Path("{id}"), and @Produces(APPLICATION_XML). The subclass wants to also produce plain text. Write the correct method override.
20. Explain why @Provider is NOT inherited through class hierarchy, unlike @Path.
21. What is the difference in JAX-RS annotation inheritance between overriding an interface default method vs a concrete class method?

Appendix — Dispatch Decision Flowchart

When an HTTP request arrives, the JAX-RS runtime follows this decision sequence:

22. Match context root → identify the WAR
23. Match `@ApplicationPath` → identify the Application and its Servlet
24. Match class-level `@Path` templates → identify candidate root resource classes
25. Match method-level `@Path` templates → identify candidate methods
26. Filter by HTTP method annotation (`@GET`, `@POST`, etc.)
27. Filter by `@Consumes` (Content-Type of request body, if present)
28. Filter by `@Produces` (Accept header of the request)
29. Apply tiebreakers (literals > variables > regex) → select one method
30. Execute the method; if it is a locator (no verb), introspect returned object and repeat from step 4

Dispatch errors and their status codes

- Step 3 fails (no class matches) → 404 Not Found
- Step 4 fails (no method path matches) → 404 Not Found
- Step 5 fails (path matches, not verb) → 405 Method Not Allowed
- Step 6 fails (Content-Type not consumed) → 415 Unsupported Media Type
- Step 7 fails (no method produces Accept) → 406 Not Acceptable